

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Tiếng Việt)

Aurélien Géron — bản dịch chỉnh sửa (2nd ed.)

Ngày 6 tháng 7 năm 2026

Mục lục

0.1	Tỷ lệ hội tụ	3
0.2	Ngẫu nhiên Gradient Descent	3
0.3	Gradient Descent lô nhỏ	4
0.4	Regression đa thức	5
0.5	Đường cong học tập	5
0.6	Sự đánh đổi bias/phương sai	6
0.7	Model tuyến tính chính quy	7
0.8	Regression sườn	7
0.9	Regression Lasso	8
0.10	Lưới đàn hồi	9
0.11	Dừng sớm	10
0.12	Regression logistic	10
0.13	Ước tính xác suất	10
0.14	Ranh giới quyết định	11
0.15	Regression Softmax	12
0.16	Entropy chéo	13
1	Bài tập	13
1.1	CHAPTER 5 Máy Vector hỗ trợ	14
1.2	Classification SVM tuyến tính	14
1.3	Classification lề mềm	15
1.4	Classification SVM phi tuyến	16
1.5	Hạt nhân đa thức	16
1.6	Tính năng tương tự	17
1.7	Hạt nhân Gaussian RBF	17
1.8	Độ phức tạp tính toán	18
1.9	Regression SVM	18
1.10	Dưới mui xe	19
1.11	Chức năng quyết định và dự đoán	19
1.12	Mục tiêu training	19
1.13	Lập trình bậc hai	20
1.14	Vấn đề kép	21
1.15	Hạt nhân SVMs	21

Hands-On Machine Learning với Scikit-Learn, Keras và TensorFlow

Sơ đồ này cũng minh họa thực tế rằng việc training model có nghĩa là tìm kiếm sự kết hợp của model parameters để giảm thiểu hàm chi phí (trên training set). Đó là một cuộc tìm kiếm trong không gian parameter của model: parameters một model càng có nhiều chiều thì không gian này càng có nhiều chiều và việc tìm kiếm càng khó: tìm kiếm một cái kim trong đồng cỏ khô 300 chiều phức tạp hơn nhiều so với tìm kiếm trong 3 chiều. May mắn thay, vì hàm chi phí là lồi trong trường hợp Linear Regression nên kim chỉ nằm ở đáy bát.

Để triển khai Gradient Descent, bạn cần tính toán gradient của hàm chi phí đối với từng model parameter j . Nói cách khác, bạn cần tính toán hàm chi phí sẽ thay đổi bao nhiêu nếu bạn thay đổi j chỉ một chút. Đây được gọi là đạo hàm riêng. Nó giống như hỏi “Gradient của ngọn núi dưới chân tôi là bao nhiêu nếu tôi quay mặt về hướng đông?” và sau đó hỏi câu hỏi tương tự hướng về phía bắc (v.v. Đối với tất cả các chiều khác, nếu bạn có thể tưởng tượng một vũ trụ có nhiều hơn ba chiều). Phương trình 4-5 tính đạo hàm riêng của hàm chi phí đối với parameter j , được ghi chú là $MSE(\theta) / \theta_j$.

Phương trình 4-5. Đạo hàm riêng của hàm chi phí

Updated for

Thay vì tính toán các đạo hàm riêng lẻ này, bạn có thể sử dụng Công thức 4-6 để tính tất cả chúng cùng một lúc. Vector gradient, được ký hiệu là $\nabla MSE(\theta)$, chứa tất cả các đạo hàm riêng của hàm chi phí (một cho mỗi model parameter).

Phương trình 4-6. Vector gradient của hàm chi phí

$$\begin{aligned} \nabla MSE &= \\ &= \begin{bmatrix} \frac{\partial MSE}{\partial \theta_0} \\ \frac{\partial MSE}{\partial \theta_1} \end{bmatrix} \end{aligned}$$

$$\begin{aligned} &= \frac{1}{n} \sum_{i=1}^n \begin{bmatrix} x_{i0} \\ x_{i1} \end{bmatrix} (y_i - \hat{y}_i) \end{aligned}$$

Lưu ý rằng công thức này bao gồm các phép tính trên toàn bộ training set X ở mỗi bước Gradient Descent! Đây là lý do tại sao thuật toán được gọi là Batch Gradient Descent: nó sử dụng toàn bộ batch của dữ liệu training ở mỗi bước (thực ra, Full Gradient Descent có lẽ sẽ là một cái tên hay hơn). Kết quả là nó cực kỳ chậm trên các tập training rất lớn (nhưng chúng ta sẽ sớm thấy các thuật toán Gradient Descent nhanh hơn nhiều). Tuy nhiên, Gradient Descent có tỷ lệ tốt với số lượng features; training Linear Regression model khi có hàng trăm nghìn features bằng cách sử dụng Gradient Descent nhanh hơn nhiều so với sử dụng Phương trình thông thường hoặc phân tích SVD.

Khi bạn đã có vector gradient hướng lên trên, bạn chỉ cần đi theo hướng ngược lại để xuống dốc. Điều này có nghĩa là trừ $\nabla MSE(\theta)$ từ θ . Đây là lúc learning rate phát huy tác dụng: nhân vector gradient với η để xác định kích thước của bước xuống dốc (Công thức 4-7).

Phương trình 4-7. Bước Gradient Descent tiếp theo $\theta = \theta - \eta \nabla MSE(\theta)$

Hãy xem cách triển khai nhanh thuật toán này:

122 | Chương 4: Training Models

Điều đó không quá khó! Hãy nhìn vào kết quả theta:

Ấn bản thứ hai

Ở bên trái, learning rate quá thấp: thuật toán cuối cùng sẽ đạt được lời giải nhưng sẽ mất nhiều thời gian. Ở phần giữa, learning rate trông khá tốt: chỉ trong vài lần lặp lại, nó đã hội tụ thành giải pháp. Ở bên phải, tốc độ học quá cao: thuật toán phân kỳ, nhảy lung tung và thực sự ngày càng rời xa lời giải ở mỗi bước.

Để tìm learning rate tốt, bạn có thể sử dụng grid search (xem Chương 2). Tuy nhiên, bạn có thể muốn giới hạn số lần lặp để grid search có thể loại bỏ models mất quá nhiều thời gian để hội tụ.

Bạn có thể thắc mắc làm thế nào để thiết lập số lần lặp. Nếu quá thấp, bạn vẫn còn rất xa giải pháp tối ưu khi thuật toán dừng lại; nhưng nếu cao quá bạn sẽ lãng phí thời gian trong khi model parameters không thay đổi nữa. Một giải pháp đơn giản là thiết lập một số lần lặp rất lớn nhưng làm gián đoạn thuật toán khi vector gradient trở nên nhỏ—nghĩa là khi chuẩn của nó trở nên nhỏ hơn một số rất nhỏ (được gọi là dung sai)—vì điều này xảy ra khi Gradient Descent đã (gần như) đạt đến mức tối thiểu.

0.1 Tỷ lệ hội tụ

0.2 Ngẫu nhiên Gradient Descent

Vấn đề chính với Batch Gradient Descent là nó sử dụng toàn bộ training set để tính toán gradient ở mỗi bước, điều này khiến nó rất chậm khi training set lớn. Ở thái cực ngược lại, Stochastic Gradient Descent chọn một phiên bản ngẫu nhiên trong training set ở mỗi bước và tính toán gradient chỉ dựa trên phiên bản đó. Rõ ràng, làm việc trên một phiên bản tại một thời điểm giúp thuật toán nhanh hơn nhiều vì nó có rất ít dữ liệu để thao tác ở mỗi lần lặp. Nó cũng giúp bạn có thể training trên các tập training khổng lồ vì chỉ cần một phiên bản trong bộ nhớ ở mỗi lần lặp (Stochastic GD có thể được triển khai như một thuật toán ngoài lõi; xem Chương 1).

Mặt khác, do tính chất ngẫu nhiên (tức là ngẫu nhiên), thuật toán này ít đều đặn hơn nhiều so với Batch Gradient Descent: thay vì giảm nhẹ cho đến khi đạt mức tối thiểu, hàm chi phí sẽ tăng lên và giảm xuống, chỉ giảm ở mức trung bình. Theo thời gian, nó sẽ tiến đến rất gần mức tối thiểu, nhưng một khi đã đạt đến mức đó, nó sẽ tiếp tục nảy lên và không bao giờ lắng xuống (xem Hình 4-9). Vì vậy, khi thuật toán dừng lại, các giá trị parameter cuối cùng sẽ tốt nhưng không tối ưu.

124 | Chương 4: Training Models

Mục lục

Do đó, tính ngẫu nhiên là tốt để thoát khỏi tối ưu cục bộ, nhưng lại là xấu vì nó có nghĩa là thuật toán không bao giờ có thể đạt được mức tối thiểu. Một giải pháp cho vấn đề nan giải này là giảm dần learning rate. Các bước bắt đầu lớn (giúp tiến bộ nhanh chóng và thoát khỏi cực tiểu cục bộ), sau đó ngày càng nhỏ hơn, cho phép thuật toán ổn định ở mức tối thiểu toàn cục. Quá trình này gần giống với quá trình ủ mô phỏng, một thuật toán được lấy cảm hứng từ quá trình ủ luyện kim, trong đó kim loại nóng chảy được làm nguội từ từ. Hàm xác định learning rate ở mỗi lần lặp được gọi là lịch học. Nếu learning rate giảm quá nhanh, bạn có thể bị kẹt ở mức tối thiểu cục bộ hoặc thậm chí bị đóng băng giữa chừng ở mức tối thiểu. Nếu learning rate giảm quá chậm, bạn có thể nhảy quanh mức tối thiểu trong một thời gian dài và đạt được giải pháp dưới mức tối ưu nếu bạn ngừng tập luyện quá sớm.

Code này triển khai Stochastic Gradient Descent bằng lịch học đơn giản:

Theo quy ước, chúng ta lặp lại theo m lần lặp; mỗi vòng được gọi là epoch. Trong khi code Batch Gradient Descent được lặp lại 1.000 lần trong toàn bộ tập training thì code này chỉ đi qua training set 50 lần và đạt được một giải pháp khá tốt:

Lưu ý rằng vì các phiên bản được chọn ngẫu nhiên nên một số phiên bản có thể được chọn nhiều lần cho mỗi epoch, trong khi những phiên bản khác có thể không được chọn chút nào. Nếu bạn muốn chắc chắn rằng thuật toán đi qua mọi phiên bản tại mỗi epoch,

một cách tiếp cận khác là xáo trộn training set (đảm bảo xáo trộn features đầu vào và các nhãn cùng nhau), sau đó duyệt qua từng phiên bản, sau đó xáo trộn lại, v.v. Tuy nhiên, cách tiếp cận này thường hội tụ chậm hơn.

Khi sử dụng Stochastic Gradient Descent, các phiên bản training phải độc lập và được phân phối giống hệt nhau (IID) để đảm bảo rằng parameters được kéo về mức tối ưu toàn cục, tính trung bình. Một cách đơn giản để đảm bảo điều này là xáo trộn các phiên bản trong quá trình training (ví dụ: chọn ngẫu nhiên từng phiên bản hoặc xáo trộn training set ở đầu mỗi epoch). Nếu bạn không xáo trộn các phiên bản—ví dụ: nếu các phiên bản được sắp xếp theo nhãn—thì SGD sẽ bắt đầu bằng cách tối ưu hóa cho một nhãn, sau đó đến nhãn tiếp theo, v.v. Và nó sẽ không đạt đến mức tối thiểu toàn cầu.

Để thực hiện Linear Regression bằng Stochastic GD với Scikit-Learn, bạn có thể sử dụng lớp SGDRegressor, lớp này mặc định tối ưu hóa hàm chi phí lỗi bình phương. Code sau chạy tối đa 1.000 epochs hoặc cho đến khi tổn thất giảm xuống dưới 0,001 trong một epoch (`max_iter=1000`, `tol=1e-3`). Nó bắt đầu với learning rate là 0,1 (`eta0=0,1`), sử dụng lịch học mặc định (khác với lịch trình trước đó). Cuối cùng, nó không sử dụng bất kỳ regularization nào (hình phạt=Không; sẽ sớm có thêm thông tin chi tiết về điều này):

126 | Chương 4: Training Models

Một lần nữa, bạn tìm thấy một nghiệm khá gần với nghiệm được trả về bởi Phương trình Chuẩn:

0.3 Gradient Descent lô nhỏ

Thuật toán Gradient Descent cuối cùng mà chúng ta sẽ xem xét có tên là Mini-batch Gradient Descent. Thật đơn giản để hiểu khi bạn biết Batch và Stochastic Gradient Descent: ở mỗi bước, thay vì tính toán gradient dựa trên training set đầy đủ (như trong Batch GD) hoặc chỉ dựa trên một phiên bản (như trong Stochastic GD), Mini-batch GD tính toán gradient trên các tập hợp phiên bản ngẫu nhiên nhỏ được gọi là mini-batches. Ưu điểm chính của Mini-batch GD so với Stochastic GD là bạn có thể tăng hiệu suất từ việc tối ưu hóa phần cứng của các hoạt động matrix, đặc biệt là khi sử dụng GPUs.

Tiến trình của thuật toán trong không gian parameter ít thất thường hơn so với Stochastic GD, đặc biệt với mini-batches khá lớn. Kết quả là, Mini-batch GD cuối cùng sẽ tiến gần đến mức tối thiểu hơn một chút so với Stochastic GD—nhưng nó có thể khó thoát khỏi mức cực tiểu cục bộ hơn (trong trường hợp các vấn đề xảy ra ở mức cực tiểu cục bộ, không giống như Linear Regression). Hình 4-11 cho thấy các đường đi của ba thuật toán Gradient Descent trong không gian parameter trong quá trình training. Tất cả đều kết thúc ở mức gần mức tối thiểu, nhưng đường đi của Batch GD thực sự dừng lại ở mức tối thiểu, trong khi cả Stochastic GD và Mini-batch GD vẫn tiếp tục đi vòng quanh. Tuy nhiên, đừng quên rằng Batch GD mất rất nhiều thời gian để thực hiện từng bước và Stochastic GD và Mini-batch GD cũng sẽ đạt mức tối thiểu nếu bạn sử dụng lịch học tốt.

Hãy so sánh các thuật toán mà chúng ta đã thảo luận cho đến nay về Regression tuyến tính (recall trong đó m là số phiên bản training và n là số features); xem Bảng 4-1.

Thuật toán Lớn m Hỗ trợ ngoài lõi Cần có siêu tham số lớn Scikit-Learn

Phương trình bình thường Nhanh Không Chậm 0 Không N/A

SVD Nhanh Không chậm 0 Không LinearRegression

Batch GD Chậm Không nhanh 2 Có SGDRegressor

Stochastic GD Nhanh Có Nhanh 2 Có SGDRegressor

Mini-batch GD Nhanh Có Nhanh 2 Có SGDRegressor

Hầu như không có sự khác biệt sau khi training: tất cả các thuật toán này đều có models rất giống nhau và đưa ra dự đoán theo cách giống hệt nhau.

0.4 Regression đa thức

Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng? Đáng ngạc nhiên là bạn có thể sử dụng model tuyến tính để khớp dữ liệu phi tuyến. Một cách đơn giản để thực hiện việc này là thêm sức mạnh của mỗi feature dưới dạng features mới, sau đó training model tuyến tính trên bộ features mở rộng này. Kỹ thuật này được gọi là Polynomial Regression.

Hãy xem một ví dụ. Trước tiên, hãy tạo một số dữ liệu phi tuyến, dựa trên phương trình bậc hai đơn giản (cộng với một số nhiễu; xem Hình 4-12):

128 | Chương 4: Training Models

X_poly hiện chứa feature ban đầu của X cộng với bình phương của feature này. Bây giờ bạn có thể lắp LinearRegression model vào dữ liệu training mở rộng này (Hình 4-13):

Không tệ: model ước tính $y = 0,56x^2 + 0,93x + 1,78$ trong khi trên thực tế, hàm ban đầu là $y = 0,5x^2 + 1,0x + 2,0$ + nhiễu Gaussian.

Lưu ý rằng khi có nhiều features, Polynomial Regression có khả năng tìm ra mối quan hệ giữa features (điều mà Linear Regression model đơn giản không thể làm được). Điều này có thể thực hiện được nhờ thực tế là PolynomialFeatures cũng bổ sung tất cả các kết hợp của features ở mức độ nhất định. Ví dụ: nếu có hai features a và b, PolynomialFeatures có độ=3 sẽ không chỉ thêm features a², a³, b² và b³ mà còn thêm các kết hợp ab, a²b và ab².

0.5 Đường cong học tập

Nếu bạn thực hiện Polynomial Regression ở mức độ cao, bạn có thể sẽ phù hợp với dữ liệu training tốt hơn nhiều so với Linear Regression đơn giản. Ví dụ: Hình 4-14 áp dụng model đa thức 300 độ cho dữ liệu training trước đó và so sánh kết quả với model tuyến tính thuần túy và model bậc hai (đa thức bậc hai). Hãy chú ý cách model đa thức 300 độ lắc lư xung quanh để đến gần nhất có thể với các trường hợp training.

130 | Chương 4: Training Models

Polynomial Regression model cấp độ cao này thực sự là overfitting dữ liệu training, trong khi model tuyến tính là underfitting nó. Model sẽ tổng quát hóa tốt nhất trong trường hợp này là model bậc hai, điều này hợp lý vì dữ liệu được tạo bằng model bậc hai. Nhưng nói chung, bạn sẽ không biết chức năng nào đã tạo ra dữ liệu, vậy làm cách nào bạn có thể quyết định mức độ phức tạp của model của mình? Làm thế nào bạn có thể biết rằng dữ liệu model của bạn là overfitting hoặc underfitting?

Trong Chương 2, bạn đã sử dụng cross-validation để ước tính hiệu suất khái quát hóa của model. Nếu model hoạt động tốt trên dữ liệu training nhưng tổng quát hóa kém theo số liệu cross-validation thì model của bạn là overfitting. Nếu nó hoạt động kém trên cả hai thì đó là underfitting. Đây là một cách để nhận biết model quá đơn giản hay quá phức tạp.

Một cách khác để nhận biết là xem xét các đường cong học tập: đây là các biểu đồ về hiệu suất của model trên training set và validation set như một hàm của kích thước training set (hoặc vòng lặp training). Để tạo sơ đồ, hãy training model nhiều lần trên các tập hợp con có kích thước khác nhau của training set. Đoạn code sau xác định một hàm, với một số dữ liệu training, vẽ đồ thị đường cong học tập của model:

Chúng ta hãy nhìn vào các đường cong học tập của Linear Regression model đơn giản (một đường thẳng; xem Hình 4-15):

Model này chính là underfitting xứng đáng được giải thích một chút. Đầu tiên, hãy xem hiệu suất trên dữ liệu training: khi chỉ có một hoặc hai trường hợp trong tập training, model có thể khớp chúng một cách hoàn hảo, đó là lý do tại sao đường cong bắt đầu từ 0. Nhưng khi các phiên bản mới được thêm vào training set, model không thể khớp hoàn hảo với dữ liệu training, vì dữ liệu bị nhiễu và vì nó hoàn toàn không tuyến tính. Vì vậy, lỗi trên dữ liệu training sẽ tăng lên cho đến khi đạt đến mức ổn định, tại thời điểm đó, việc thêm các phiên bản mới vào training set không làm cho lỗi trung bình trở nên tốt hơn hay tệ hơn nhiều. Bây giờ chúng ta hãy xem hiệu suất của model trên dữ liệu xác thực. Khi model được training trên rất ít trường hợp training, nó không có khả năng khái quát hóa đúng cách, đó là lý do tại sao lỗi xác thực ban đầu khá lớn. Sau đó, với tư cách là

132 | Chương 4: Training Models

Model được hiển thị nhiều hơn training examples, nó học hỏi và do đó lỗi xác thực sẽ giảm dần. Tuy nhiên, một lần nữa, một đường thẳng không thể thực hiện tốt công việc model hóa dữ liệu, do đó, lỗi kết thúc ở mức ổn định, rất gần với đường cong kia.

Những đường cong học tập này là điển hình của model đó là underfitting. Cả hai đường cong đã đạt đến một điểm bằng phẳng; chúng gần nhau và khá cao.

Nếu model của bạn là underfitting thì dữ liệu training thì việc thêm nhiều training example sẽ không giúp ích gì. Bạn cần sử dụng model phức tạp hơn hoặc tìm ra features tốt hơn.

Bây giờ chúng ta hãy xem đường cong học tập của đa thức model bậc 10 trên cùng một dữ liệu (Hình 4-16):

Những đường cong học tập này trông hơi giống những đường cong trước, nhưng có hai điểm khác biệt rất quan trọng:

- Lỗi trên dữ liệu training thấp hơn nhiều so với Linear Regression model.
- Có khoảng cách giữa các đường cong. Điều này có nghĩa là model hoạt động trên dữ liệu training tốt hơn đáng kể so với dữ liệu xác thực, đây là dấu hiệu feature của overfitting model. Tuy nhiên, nếu bạn sử dụng training set lớn hơn nhiều, hai đường cong sẽ tiếp tục gần nhau hơn.

Một cách để cải thiện overfitting model là cung cấp thêm dữ liệu training cho nó cho đến khi lỗi xác thực đạt tới training error.

0.6 Sự đánh đổi bias/phương sai

Một kết quả lý thuyết quan trọng của thống kê và Machine Learning là lỗi tổng quát hóa của model có thể được biểu thị dưới dạng tổng của ba lỗi rất khác nhau:

Bias Phần lỗi tổng quát hóa này là do các giả định sai, chẳng hạn như giả sử rằng dữ liệu là tuyến tính khi nó thực sự là bậc hai. Bias model cao có nhiều khả năng không phù hợp với dữ liệu training.

Variance Phần này là do độ nhạy quá mức của model đối với những thay đổi nhỏ trong dữ liệu training. Model có nhiều bậc tự do (chẳng hạn như model đa thức bậc cao) có thể có variance cao và do đó phù hợp quá mức với dữ liệu training.

Lỗi không thể sửa chữa Phần này là do bản thân dữ liệu bị nhiễu. Cách duy nhất để giảm phần lỗi này là xóa dữ liệu (ví dụ: sửa nguồn dữ liệu, chẳng hạn như cảm biến bị hỏng hoặc phát hiện và loại bỏ các giá trị ngoại lệ).

Việc tăng độ phức tạp của model thường sẽ làm tăng variance và giảm bias của nó. Ngược lại, việc giảm độ phức tạp của model sẽ làm tăng bias và giảm variance của nó. Đây là lý do tại sao nó được gọi là sự đánh đổi.

0.7 Model tuyến tính chính quy

Như chúng ta đã thấy trong Chương 1 và 2, một cách tốt để giảm overfitting là hợp thức hóa model (tức là hạn chế nó): càng có ít bậc tự do thì càng khó thực hiện

134 | Chương 4: Training Models

Để nó phù hợp quá mức với dữ liệu. Một cách đơn giản để chuẩn hóa model đa thức là giảm số bậc đa thức.

Đối với model tuyến tính, regularization thường đạt được bằng cách ràng buộc weights của model. Bây giờ chúng ta sẽ xem xét Ridge Regression, Lasso Regression và Elastic Net, chúng triển khai ba cách khác nhau để hạn chế weights.

0.8 Regression sườn

Ridge Regression (còn gọi là Tikhonov regularization) là phiên bản chuẩn hóa của Linear Regression: một số hạng regularization bằng $\lambda \sum_{i=1}^n w_i^2$ được thêm vào hàm chi phí. Điều này buộc thuật toán học không chỉ phù hợp với dữ liệu mà còn giữ cho model weights nhỏ nhất có thể. Lưu ý rằng thuật ngữ regularization chỉ nên được thêm vào hàm chi phí trong quá trình training. Sau khi model được training, bạn muốn sử dụng thước đo hiệu suất không chính quy để đánh giá hiệu suất của model.

Điều khá phổ biến là hàm chi phí được sử dụng trong quá trình training khác với thước đo hiệu suất được sử dụng để kiểm tra. Ngoài regularization, một lý do khác khiến chúng có thể khác nhau là hàm chi phí training tốt phải có các dẫn xuất thân thiện với việc tối ưu hóa, trong khi thước đo hiệu suất được sử dụng để thử nghiệm phải càng gần với mục tiêu cuối cùng càng tốt. Ví dụ: các classifier thường được training bằng cách sử dụng hàm chi phí chẳng hạn như mất log (sẽ được thảo luận sau) nhưng được đánh giá bằng precision/recall.

Hyperparameter kiểm soát mức độ bạn muốn thường xuyên hóa model. Nếu $\lambda = 0$ thì Ridge Regression chỉ là Linear Regression. Nếu λ rất lớn thì tất cả weights cuối cùng sẽ rất gần bằng 0 và kết quả là một đường thẳng đi qua giá trị trung bình của dữ liệu. Phương trình 4-8 trình bày hàm chi phí Ridge Regression.⁹

Phương trình 4-8. Hàm chi phí Ridge Regression

$$J = \text{MSE} + \frac{\lambda}{2} \sum_{i=1}^n w_i^2$$

Lưu ý rằng số hạng bias 0 không được chuẩn hóa (tổng bắt đầu từ $i = 1$, không phải 0). Nếu chúng ta định nghĩa w là vector của feature weights (1 đến n), thì số hạng regularization là

Bằng $\frac{\lambda}{2} (w^T w)$, trong đó $w^T w$ đại diện cho chuẩn 2 của vector weight.¹⁰ Đối với Gradient Descent, chỉ cần thêm w vào vector gradient MSE (Phương trình 4-6).

Điều quan trọng là phải chia tỷ lệ dữ liệu (ví dụ: sử dụng StandardScaler) trước khi thực hiện Ridge Regression, vì nó rất nhạy cảm với tỷ lệ của features đầu vào. Điều này đúng với hầu hết models được chuẩn hóa.

Giống như Linear Regression, chúng ta có thể thực hiện Ridge Regression bằng cách tính phương trình dạng đóng hoặc bằng cách thực hiện Gradient Descent. Ưu và nhược điểm là

136 | Chương 4: Training Models

Như nhau. Phương trình 4-9 cho thấy nghiệm dạng đóng, trong đó A là matrix đồng nhất $(n + 1) \times (n + 1)$, ngoại trừ số 0 ở ô trên cùng bên trái, tương ứng với số hạng bias.

Phương trình 4-9. Giải pháp dạng đóng Ridge Regression
$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Đây là cách thực hiện Ridge Regression với Scikit-Learn bằng cách sử dụng giải pháp dạng đóng (một biến thể của Phương trình 4-9 sử dụng kỹ thuật phân tích nhân tử matrix của AndréLouis Cholesky):

Và sử dụng Stochastic Gradient Descent:

Hình phạt hyperparameter đặt ra loại thuật ngữ regularization sẽ sử dụng. Việc chỉ định "l2" cho biết rằng bạn muốn SGD thêm số hạng regularization vào hàm chi phí bằng một nửa bình phương của định mức λ của vector weight: đây đơn giản là Ridge Regression.

0.9 Regression Lasso

Toán tử lựa chọn và co rút tuyệt đối nhỏ nhất Regression (thường được gọi đơn giản là Lasso Regression) là một phiên bản chính quy khác của Linear Regression: giống như Ridge Regression, nó thêm số hạng regularization vào hàm chi phí, nhưng nó sử dụng định mức λ của vector weight thay vì một nửa bình phương của định mức λ (xem Công thức 4-10).

Phương trình 4-10. Hàm chi phí Lasso Regression

$$J = \text{MSE} + \lambda \sum_{i=1}^n |\beta_i|$$

Một đặc điểm quan trọng của Lasso Regression là nó có xu hướng loại bỏ weights của features ít quan trọng nhất (tức là đặt chúng về 0). Ví dụ, đường đứt nét trong biểu đồ bên phải trong Hình 4-18 (với $\lambda = 10^{-7}$) trông có vẻ bậc hai, gần như tuyến tính: tất cả weights cho đa thức bậc cao features đều bằng 0. Nói cách khác, Lasso Regression tự động thực hiện lựa chọn feature và xuất ra một model thừa thớt (tức là có một vài feature weights khác 0).

Bạn có thể hiểu lý do tại sao lại như vậy bằng cách xem Hình 4-19: các trục đại diện cho hai model parameters và các đường viền nền biểu thị các hàm mất mát khác nhau. Trong biểu đồ trên cùng bên trái, các đường viền biểu thị tổn thất $J_1 (|\beta_1| + |\beta_2|)$, giảm tuyến tính khi bạn đến gần bất kỳ trục nào. Ví dụ: nếu bạn khởi tạo model parameters thành $\beta_1 = 2$ và $\beta_2 = 0,5$, việc chạy Gradient Descent sẽ giảm cả hai parameters như nhau (như được biểu thị bằng đường đứt nét màu vàng); do đó β_2 sẽ về 0 trước tiên (vì ban đầu nó gần với 0 hơn). Sau đó, Gradient Descent sẽ lăn xuống máng xối cho đến khi đạt $\beta_1 = 0$ (với một chút nảy lên, vì gradient của J_1 không bao giờ gần bằng 0: chúng là -1 hoặc 1 cho mỗi parameter). Trong biểu đồ bên phải, các đường viền biểu thị hàm chi phí của Lasso (tức là hàm chi phí MSE cộng với tổn thất J_1). Các vòng tròn nhỏ màu trắng hiển thị đường dẫn mà Gradient Descent sử dụng để tối ưu hóa một số model parameters đã được khởi tạo xung quanh $\beta_1 = 0,25$ và $\beta_2 = -1$: hãy chú ý một lần nữa rằng đường dẫn nhanh chóng đạt đến $\beta_2 = 0$, sau đó cuộn xuống máng xối và cuối cùng nảy xung quanh điểm tối ưu toàn cục (được biểu thị bằng hình vuông màu đỏ). Nếu chúng ta tăng λ , mức tối ưu toàn cục sẽ di chuyển sang trái dọc theo đường đứt nét màu vàng, trong khi

138 | Chương 4: Training Models

Nếu chúng ta giảm λ , mức tối ưu toàn cục sẽ di chuyển sang phải (trong ví dụ này, parameters tối ưu cho MSE không chính quy là $\beta_1 = 2$ và $\beta_2 = 0,5$).

Hai ô dưới cùng cho thấy điều tương tự nhưng thay vào đó có hình phạt J_2 . Trong biểu

đồ phía dưới bên trái, bạn có thể thấy rằng tổn thất J giảm theo khoảng cách đến điểm gốc, do đó Gradient Descent chỉ đi một đường thẳng tới điểm đó. Trong biểu đồ phía dưới bên phải, các đường viền biểu thị hàm chi phí của Ridge Regression (tức là hàm chi phí MSE cộng với tổn thất λ). Có hai điểm khác biệt chính với Lasso. Đầu tiên, gradient trở nên nhỏ hơn khi parameters đạt đến mức tối ưu toàn cục, do đó Gradient Descent tự nhiên chậm lại, điều này giúp hội tụ (vì không có hiện tượng nảy xung quanh). Thứ hai, parameters tối ưu (được biểu thị bằng hình vuông màu đỏ) ngày càng tiến gần đến điểm gốc khi bạn tăng λ , nhưng chúng không bao giờ bị loại bỏ hoàn toàn.

Để tránh Gradient Descent nảy quanh mức tối ưu khi sử dụng Lasso, bạn cần giảm dần tốc độ học trong quá trình training (nó vẫn sẽ nảy quanh mức tối ưu, nhưng các bước sẽ ngày càng nhỏ hơn nên sẽ hội tụ).

Hàm chi phí Lasso không khả vi tại $w_i = 0$ (với $i = 1, 2, \dots, n$), nhưng Gradient Descent vẫn hoạt động tốt nếu bạn sử dụng vector gradient phụ g13 thay vào đó khi bất kỳ $w_i = 0$ nào. Phương trình 4-11 hiển thị phương trình vector gradient phụ mà bạn có thể sử dụng cho Gradient Descent với hàm chi phí Lasso.

Phương trình 4-11. Lasso Regression vector gradient con g , $J = \text{MSE} + \text{dấu} \cdot \lambda$

dấu n trong đó dấu $i =$
 -1 nếu $w_i < 0$
 0 nếu $w_i = 0$
 $+1$ nếu $w_i > 0$

Đây là một ví dụ Scikit-Learn nhỏ sử dụng lớp Lasso:

Lưu ý rằng thay vào đó bạn có thể sử dụng `SGDRegressor(penalty="l1")`.

0.10 Lưới đàn hồi

Elastic Net là điểm trung gian giữa Ridge Regression và Lasso Regression. Thuật ngữ regularization là sự kết hợp đơn giản của cả thuật ngữ regularization của Ridge và Lasso, đồng thời bạn có thể kiểm soát tỷ lệ kết hợp r . Khi $r = 0$, Elastic Net tương đương với Ridge Regression, và khi $r = 1$, nó tương đương với Lasso Regression (xem Công thức 4-12).

Phương trình 4-12. Hàm chi phí Elastic Net

$$J = \text{MSE} + r \sum_{i=1}^n |w_i| + (1-r) \sum_{i=1}^n w_i^2$$

Vậy khi nào bạn nên sử dụng Linear Regression đơn giản (tức là không có bất kỳ regularization), Ridge, Lasso hoặc Elastic Net nào? Hầu như luôn luôn tốt hơn nếu có ít nhất một chút regularization, vì vậy nhìn chung bạn nên tránh Linear Regression đơn giản. Ridge là một mặc định tốt, nhưng nếu bạn nghi ngờ rằng chỉ một số features hữu ích, bạn nên ưu tiên Lasso hoặc Elastic Net vì chúng có xu hướng giảm features' weights vô dụng xuống 0, như chúng ta đã thảo luận. Nói chung, Elastic Net được ưa thích hơn Lasso vì Lasso

140 | Chương 4: Training Models

Có thể hoạt động thất thường khi số lượng features lớn hơn số lượng phiên bản training hoặc khi một số features có mối tương quan chặt chẽ với nhau.

Dưới đây là một ví dụ ngắn sử dụng ElasticNet của Scikit-Learn (`l1_ratio` tương ứng với tỷ lệ trộn r):

0.11 Dừng sớm

Một cách rất khác để chính quy hóa các thuật toán học lặp như Gradient Descent là dừng training ngay khi lỗi xác thực đạt đến mức tối thiểu. Cái này được gọi là early stopping. Hình 4-20 cho thấy một model phức tạp (trong trường hợp này là Polynomial Regression model cấp độ cao) đang được training với Batch Gradient Descent. Khi epochs thực hiện theo thuật toán, lỗi dự đoán của nó (RMSE) trên training set sẽ giảm, cùng với lỗi dự đoán của nó trên validation set. Tuy nhiên, sau một thời gian, lỗi xác thực sẽ ngừng giảm và bắt đầu quay trở lại. Điều này cho thấy model đã bắt đầu điều chỉnh quá mức dữ liệu training. Với early stopping, bạn chỉ cần dừng training ngay khi lỗi xác thực đạt đến mức tối thiểu. Kỹ thuật regularization đơn giản và hiệu quả đến nỗi Geoffrey Hinton gọi nó là “bữa trưa tuyệt vời miễn phí”.

Với Stochastic và Mini-batch Gradient Descent, các đường cong không được mượt mà và có thể khó biết liệu bạn đã đạt đến mức tối thiểu hay chưa. Một giải pháp là chỉ dừng lại sau khi lỗi xác thực đã vượt quá mức tối thiểu trong một thời gian (khi bạn tin chắc rằng model sẽ không hoạt động tốt hơn), sau đó khôi phục model parameters về điểm có lỗi xác thực ở mức tối thiểu.

Đây là cách triển khai cơ bản của early stopping:

0.12 Regression logistic

Như chúng ta đã thảo luận trong Chương 1, một số thuật toán regression có thể được sử dụng để classification (và ngược lại). Logistic Regression (còn gọi là Logit Regression) thường được sử dụng để ước tính xác suất một phiên bản thuộc về một lớp cụ thể (ví dụ: xác suất email này là thư rác là bao nhiêu?). Nếu xác suất ước tính lớn hơn 50% thì model dự đoán rằng individual đó thuộc về lớp đó (được gọi là lớp dương, được gắn nhãn “1”), và ngược lại, nó dự đoán rằng nó không thuộc về lớp đó (tức là nó thuộc về lớp âm, được gắn nhãn “0”). Điều này làm cho nó trở thành một classifier nhị phân.

142 | Chương 4: Training Models

0.13 Ước tính xác suất

Vậy Logistic Regression hoạt động như thế nào? Giống như Linear Regression model, Logistic Regression model tính tổng có weight của features đầu vào (cộng với số hạng bias), nhưng thay vì xuất kết quả trực tiếp như Linear Regression model, nó xuất ra logistic của kết quả này (xem Công thức 4-13).

Phương trình 4-13. Logistic Regression model xác suất ước tính (dạng vector hóa) $p = h(x) = \sigma(x)$

Logistic—được ký hiệu là $\sigma(\cdot)$ —là một hàm sigmoid (tức là hình chữ S) tạo ra một số từ 0 đến 1. Nó được định nghĩa như thể hiện trong Công thức 4-14 và Hình 4-21.

Phương trình 4-14. Hàm logistic $\sigma(t) = \frac{1}{1 + \exp(-t)}$

Khi Logistic Regression model đã ước tính xác suất $p = h(x)$ mà một thể hiện x thuộc về lớp dương, nó có thể đưa ra dự đoán $\hat{y}$ dễ dàng (xem Công thức 4-15).

Phương trình 4-15. Dự đoán Logistic Regression model $y = 0$ nếu $p < 0,5$

1 nếu $p \geq 0,5$

Lưu ý rằng $\sigma(t) < 0,5$ khi $t < 0$ và $\sigma(t) \geq 0,5$ khi $t \geq 0$, do đó Logistic Regression model dự đoán 1 nếu x dương và 0 nếu nó âm.

Điểm t thường được gọi là logit. Tên này xuất phát từ thực tế là hàm logit, được định nghĩa là $\text{logit}(p) = \log(p / (1 - p))$, là nghịch đảo của hàm logistic. Thật vậy, nếu bạn tính logit của xác suất ước tính p , bạn sẽ thấy kết quả là t . Logit còn được gọi là log-odds, vì nó là log của tỷ lệ giữa xác suất ước tính cho lớp dương và xác suất ước tính cho lớp âm.

Chức năng training và chi phí

Bây giờ bạn đã biết Logistic Regression model ước tính xác suất và đưa ra dự đoán như thế nào. Nhưng nó được training như thế nào? Mục tiêu của việc training là đặt vector parameter sao cho model ước tính xác suất cao cho các trường hợp dương ($y = 1$) và xác suất thấp cho các trường hợp âm ($y = 0$). Ý tưởng này được thể hiện bằng hàm chi phí được hiển thị trong Công thức 4-16 cho một trường hợp training x .

Phương trình 4-16. Hàm chi phí của một individual training duy nhất $c =$

Hàm chi phí trên toàn bộ training set là chi phí trung bình trên tất cả các trường hợp training. Nó có thể được viết bằng một biểu thức duy nhất gọi là log loss, được hiển thị trong phương trình 4-17.

Phương trình 4-17. Hàm chi phí Logistic Regression (mất log)

$$J = -\frac{1}{m} \sum_{i=1}^m y_i \log p_i + (1 - y_i) \log (1 - p_i)$$

Tin xấu là không có phương trình dạng đóng nào được biết đến để tính giá trị của giúp tối thiểu hóa hàm chi phí này (không có phương trình nào tương đương với Phương trình Chuẩn). Tin tốt là hàm chi phí này lồi, do đó Gradient Descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) được đảm bảo tìm ra mức tối thiểu toàn cục (nếu việc học

144 | Chương 4: Training Models

Tỷ lệ không quá lớn và bạn phải đợi đủ lâu). Đạo hàm riêng của hàm chi phí đối với model parameter θ_j thứ j được cho bởi Công thức 4-18.

Phương trình 4-18. Hàm chi phí logistic đạo hàm riêng phần

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (x_i - y_i) x_{ij}$$

Phương trình này trông rất giống Phương trình 4-5: với mỗi trường hợp, nó tính toán sai số dự đoán và nhân nó với giá trị feature thứ j , sau đó nó tính giá trị trung bình trên tất cả các trường hợp training. Khi bạn có vector gradient chứa tất cả các đạo hàm riêng, bạn có thể sử dụng nó trong thuật toán Batch Gradient Descent. Vậy là xong: bây giờ bạn đã biết cách training Logistic Regression model. Đối với Stochastic GD, bạn sẽ lấy từng phiên bản một và đối với Mini-batch GD, bạn sẽ sử dụng một lô nhỏ mỗi lần.

0.14 Ranh giới quyết định

Hãy sử dụng một dataset để minh họa Logistic Regression. Đây là một dataset nổi tiếng có chiều dài và chiều rộng dài hoa và cánh hoa của 150 hoa diên vĩ thuộc ba loài khác nhau: Iris setosa, Iris versicolor và Iris virginica (xem Hình 4-22).

Hãy thử xây dựng một classifier để phát hiện loại Iris virginica chỉ dựa trên chiều rộng cánh hoa feature. Đầu tiên hãy tải dữ liệu:

Bây giờ hãy training Logistic Regression model:

Hãy xem xác suất ước tính của model đối với những bông hoa có chiều rộng cánh hoa thay đổi từ 0 cm đến 3 cm (Hình 4-23):

146 | Chương 4: Training Models

Lớp học). Ở giữa những thái cực này, classifier không chắc chắn. Tuy nhiên, nếu bạn yêu cầu nó dự đoán lớp (sử dụng phương thức `predict_proba()` thay vì phương thức `predict()`), nó sẽ trả về lớp nào có nhiều khả năng xảy ra nhất. Do đó, có một ranh giới quyết định ở khoảng 1,6 cm trong đó cả hai xác suất đều bằng 50%: nếu chiều

rộng cánh hoa cao hơn 1,6 cm, classifier sẽ dự đoán rằng bông hoa đó là Iris virginica, và nếu không, nó sẽ dự đoán rằng đó không phải là hoa Iris virginica (ngay cả khi nó không chắc chắn lắm):

Cũng giống như models tuyến tính khác, Logistic Regression models có thể được chính quy hóa bằng cách sử dụng hình phạt 1 hoặc 2. Scikit-Learn thực sự thêm một hình phạt 2 theo mặc định.

Hyperparameter kiểm soát cường độ regularization của Scikit-Learn LogisticRegression model không phải là alpha (như trong models tuyến tính khác), mà là nghịch đảo của nó: C. Giá trị của C càng cao thì model càng ít được chuẩn hóa.

0.15 Regression Softmax

Logistic Regression model có thể được khái quát hóa để hỗ trợ trực tiếp nhiều lớp mà không cần phải training và kết hợp nhiều classifier nhị phân (như đã thảo luận trong Chương 3). Điều này được gọi là Softmax Regression hoặc Logistic Regression đa thức.

Phương trình 4-19. Điểm softmax cho lớp k sk x = x k

Lưu ý rằng mỗi lớp có vector parameter chuyên dụng (k). Tất cả các vector này thường được lưu dưới dạng hàng trong matrix parameter Θ .

Khi bạn đã tính điểm của mỗi lớp cho phiên bản x, bạn có thể ước tính xác suất pk mà phiên bản đó thuộc về lớp k bằng cách chạy điểm thông qua hàm softmax (Công thức 4-20). Hàm tính toán số mũ của mỗi điểm, sau đó chuẩn hóa chúng (chia cho tổng của tất cả các số mũ). Điểm số thường được gọi là logit hoặc log-odds (mặc dù thực tế chúng là log-odds không chuẩn hóa).

Phương trình 4-20. Hàm softmax
$$p_k = \frac{s_k x}{\sum_{j=1}^K \exp s_j x}$$

Trong phương trình này:

- K là số lớp.
- $s(x)$ là một vector chứa điểm của mỗi lớp đối với instance x.
- $(s(x))_k$ là xác suất ước tính để thể hiện x thuộc về lớp k, với điểm số của mỗi lớp đối với thể hiện đó.

148 | Chương 4: Training Models

Giống như classifier Logistic Regression, classifier Softmax Regression dự đoán lớp có xác suất ước tính cao nhất (đơn giản là lớp có điểm cao nhất), như được hiển thị trong Phương trình 4-21.

Phương trình 4-21. Dự đoán classification Softmax Regression
$$y = \operatorname{argmax}_k s_k x = \operatorname{argmax}_k s_k x$$

Toán tử argmax trả về giá trị của biến tối đa hóa hàm. Trong phương trình này, nó trả về giá trị của k để tối đa hóa xác suất ước tính $(s(x))_k$.

Bây giờ bạn đã biết cách model ước tính xác suất và đưa ra dự đoán, hãy xem quá trình training. Mục tiêu là có model ước tính xác suất cao cho lớp mục tiêu (và do đó xác suất thấp cho các lớp khác). Việc giảm thiểu hàm chi phí được hiển thị trong Công thức 4-22, được gọi là entropy chéo, sẽ dẫn đến mục tiêu này vì nó sẽ phạt model khi nó ước tính xác suất thấp cho một lớp mục tiêu. Entropy chéo thường được sử dụng để đo mức độ phù hợp của một tập hợp các xác suất của lớp ước tính với các lớp mục tiêu.

Phương trình 4-22. Hàm chi phí entropy chéo

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{k,i} \log p_{k,i}$$

Trong phương trình này:

- $y_{k,i}$ là xác suất mục tiêu mà individual thứ i thuộc lớp k . Nói chung, nó bằng 1 hoặc 0, tùy thuộc vào việc thể hiện có thuộc lớp hay không.

Lưu ý rằng khi chỉ có hai lớp ($K = 2$), hàm chi phí này tương đương với hàm chi phí của Logistic Regression (log loss; xem Công thức 4-17).

0.16 Entropy chéo

Entropy chéo có nguồn gốc từ lý thuyết thông tin. Giả sử bạn muốn truyền tải thông tin thời tiết hàng ngày một cách hiệu quả. Nếu có tám tùy chọn (nắng, mưa, v.v.), bạn có thể code hóa mỗi tùy chọn bằng ba bit vì $2^3 = 8$. Tuy nhiên, nếu bạn cho rằng hầu như ngày nào trời cũng nắng, thì sẽ hiệu quả hơn nhiều khi code hóa “nắng” chỉ trên một bit (0) và bảy tùy chọn còn lại trên bốn bit (bắt đầu bằng 1). Entropy chéo đo số bit trung bình bạn thực sự gửi cho mỗi tùy chọn. Nếu giả định của bạn về thời tiết là hoàn hảo thì entropy chéo sẽ bằng entropy của chính thời tiết (tức là tính khó dự đoán nội tại của nó). Nhưng nếu giả định của bạn sai (ví dụ: nếu trời thường mưa), thì entropy chéo sẽ lớn hơn một lượng gọi là phân kỳ Kullback–Leibler (KL).

Entropy chéo giữa hai phân bố xác suất p và q được định nghĩa là $H(p,q) = -\sum_x p(x) \log q(x)$ (ít nhất là khi các phân bố rời rạc). Để biết thêm chi tiết, hãy xem video của tôi về chủ đề này.

Vector gradient của hàm chi phí này đối với (k) được cho bởi Công thức 4-23.

Phương trình 4-23. Vector gradient entropy chéo cho lớp k

$$k \nabla J_{\Theta} = \frac{1}{m} \sum_{i=1}^m p_{k,i} - y_{k,i} x_i$$

Bây giờ bạn có thể tính toán vector gradient cho mọi lớp, sau đó sử dụng Gradient Descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) để tìm matrix parameter Θ giúp giảm thiểu hàm chi phí.

Hãy sử dụng Softmax Regression để classification hoa diên vĩ thành cả ba lớp. Theo mặc định, LogisticRegression của ScikitLearn sử dụng chế độ một so với phần còn lại khi bạn training nó trên nhiều hơn hai lớp, nhưng bạn có thể đặt `multi_class` hyperparameter thành “multinomial” để chuyển nó sang Softmax Regression. Bạn cũng phải chỉ định một bộ giải hỗ trợ Softmax Regression, chẳng hạn như bộ giải “lbfgs” (xem tài liệu của Scikit-Learn để biết thêm chi tiết). Nó cũng áp dụng L_2 regularization theo mặc định mà bạn có thể kiểm soát bằng hyperparameter C :

150 | Chương 4: Training Models

1 Bài tập

1. Bạn có thể sử dụng thuật toán training Linear Regression nào nếu bạn có training set với hàng triệu features?

2. Giả sử features trong training set của bạn có tỷ lệ rất khác nhau. Những thuật toán nào có thể gặp phải vấn đề này và bằng cách nào? Bạn có thể làm gì về nó?

3. Gradient Descent có thể bị kẹt ở mức tối thiểu cục bộ khi training Logistic Regression model không?

4. Có phải tất cả các thuật toán Gradient Descent đều dẫn đến cùng một model, miễn là bạn để chúng chạy đủ lâu?

5. Giả sử bạn sử dụng Batch Gradient Descent và bạn vẽ biểu đồ lỗi xác thực ở mọi epoch. Nếu bạn nhận thấy lỗi xác thực liên tục tăng lên thì điều gì có thể xảy ra? Làm thế nào bạn có thể khắc phục điều này?

6. Có nên dừng Mini-batch Gradient Descent ngay lập tức khi lỗi xác thực xuất hiện không?

7. Thuật toán Gradient Descent nào (trong số những thuật toán chúng ta đã thảo luận) sẽ đạt đến vùng lân cận của giải pháp tối ưu nhanh nhất? Cái nào thực sự sẽ hội tụ? Làm thế nào bạn có thể làm cho những người khác cũng hội tụ?

8. Giả sử bạn đang sử dụng Polynomial Regression. Bạn vẽ đường cong học tập và nhận thấy rằng có một khoảng cách lớn giữa training error và lỗi xác thực. Chuyện gì đang xảy ra vậy? Ba cách để giải quyết điều này là gì?

9. Giả sử bạn đang sử dụng Ridge Regression và bạn nhận thấy rằng training error và lỗi xác thực gần như bằng nhau và khá cao. Bạn cho rằng model bị bias cao hay variance cao? Bạn nên tăng hay giảm độ chính quy hóa hyperparameter ?

10. Tại sao bạn muốn sử dụng:

Một. Ridge Regression thay vì Linear Regression đơn giản (tức là không có bất kỳ quy định nào)?

B. Lasso thay vì Ridge Regression?

C. Elastic Net thay vì Lasso?

11. Giả sử bạn muốn classification hình ảnh thành ngoài trời/trong nhà và ban ngày/đêm. Bạn nên triển khai hai classifier Logistic Regression hay một classifier regression Softmax?

12. Triển khai Batch Gradient Descent với early stopping cho Softmax Regression (không sử dụng Scikit-Learn).

Lời giải cho các bài tập này có sẵn trong Phụ lục A.

152 | Chương 4: Training Models

1.1 CHAPTER 5 Máy Vector hỗ trợ

Support Vector Machine (SVM) là Machine Learning model mạnh mẽ và linh hoạt, có khả năng thực hiện classification tuyến tính hoặc phi tuyến tính, regression và thậm chí cả phát hiện ngoại lệ. Đây là một trong những models phổ biến nhất trong Machine Learning và bất kỳ ai quan tâm đến Machine Learning nên có nó trong hộp công cụ của họ. SVMs đặc biệt phù hợp với classification của datasets cỡ nhỏ hoặc vừa phức tạp.

Chương này sẽ giải thích các khái niệm cốt lõi của SVMs, cách sử dụng và cách chúng hoạt động.

1.2 Classification SVM tuyến tính

Ý tưởng cơ bản đằng sau SVMs được giải thích rõ nhất bằng một số hình ảnh. Hình 5-1 cho thấy một phần của mớng mắt dataset đã được giới thiệu ở cuối Chương 4. Hai lớp này có thể được phân tách rõ ràng dễ dàng bằng một đường thẳng (chúng có thể phân tách tuyến tính). Biểu đồ bên trái hiển thị ranh giới quyết định của ba classification tuyến tính có thể có. Model có ranh giới quyết định được biểu thị bằng đường đứt nét tệ đến mức nó thậm chí không phân tách các lớp một cách chính xác. Hai models còn lại hoạt động hoàn hảo trên training set này, nhưng ranh giới quyết định của chúng quá gần với các phiên bản nên models này có thể sẽ không hoạt động tốt trên các phiên bản mới. Ngược lại, đường liền nét trong biểu đồ bên phải biểu thị ranh giới quyết định của classifier SVM; đường này không chỉ ngăn cách hai lớp mà còn ở càng xa các trường training gần nhất càng tốt. Bạn có thể coi classifier SVM phù hợp với đường phổ rộng nhất có thể (được

biểu thị bằng các đường đứt nét song song) giữa các lớp. Đây được gọi là classification có biên độ lớn.

Lưu ý rằng việc thêm nhiều trường hợp training “ngoài đường” sẽ không ảnh hưởng đến ranh giới quyết định chút nào: nó hoàn toàn được xác định (hoặc “được hỗ trợ”) bởi các trường hợp nằm ở rìa đường. Những trường hợp này được gọi là các vector hỗ trợ (chúng được khoanh tròn trong Hình 5-1).

SVMs rất nhạy cảm với tỷ lệ feature, như bạn có thể thấy trong Hình 5-2: trong biểu đồ bên trái, tỷ lệ dọc lớn hơn nhiều so với tỷ lệ ngang, do đó đường rộng nhất có thể gần với tỷ lệ ngang. Sau khi chia tỷ lệ feature (ví dụ: sử dụng StandardScaler của Scikit-Learn), ranh giới quyết định trong biểu đồ bên phải trông đẹp hơn nhiều.

1.3 Classification lề mềm

Nếu chúng ta áp đặt nghiêm ngặt rằng tất cả các phiên bản phải ở ngoài đường và ở phía bên phải, thì đây được gọi là lề cứng classification. Có hai vấn đề chính với việc classification lề cứng. Đầu tiên, nó chỉ hoạt động nếu dữ liệu có thể phân tách tuyến tính. Thứ hai, nó nhạy cảm với các ngoại lệ. Hình 5-3 cho thấy móng mắt dataset chỉ có một ngoại lệ bổ sung: ở bên trái, không thể tìm thấy lề cứng; ở bên phải, ranh giới quyết định sẽ rất khác so với ranh giới chúng ta thấy trong Hình 5-1 nếu không có ngoại lệ, và nó có lề cũng sẽ không khái quát hóa.

154 | Chương 5: Máy vector hỗ trợ

Để tránh những vấn đề này, hãy sử dụng model linh hoạt hơn. Mục tiêu là tìm ra sự cân bằng tốt giữa việc giữ cho đường càng rộng càng tốt và hạn chế vi phạm lề đường (tức là các trường hợp dừng ở giữa đường hoặc thậm chí đi sai phía). Điều này được gọi là lề mềm classification.

Khi tạo SVM model bằng Scikit-Learn, chúng ta có thể chỉ định một số siêu parameters. C là một trong những hyperparameters đó. Nếu chúng ta đặt nó ở giá trị thấp thì chúng ta sẽ có model ở bên trái Hình 5-4. Với giá trị cao, chúng ta có model ở bên phải. Vi phạm ký quỹ là xấu. Thông thường sẽ tốt hơn nếu có một vài trong số chúng. Tuy nhiên, trong trường hợp này model ở bên trái có nhiều vi phạm về lề nhưng có lẽ sẽ khái quát hóa tốt hơn.

Nếu SVM model của bạn là overfitting, bạn có thể thử điều chỉnh nó bằng cách giảm C .

Code Scikit-Learn sau đây tải dataset móng mắt, chia tỷ lệ features, sau đó training SVM model tuyến tính (sử dụng lớp LinearSVC với $C=1$ và bản lề loss hàm, được mô tả ngắn gọn) để phát hiện hoa Iris virginica:

Model thu được được thể hiện ở bên trái trong Hình 5-4.

Sau đó, như thường lệ, bạn có thể sử dụng model để đưa ra dự đoán:

Không giống như classifier Logistic Regression, classifier SVM không đưa ra xác suất cho mỗi lớp.

Thay vì sử dụng lớp LinearSVC, chúng ta có thể sử dụng lớp SVC với nhân tuyến tính. Khi tạo SVC model, chúng ta sẽ viết SVC(kernel="tuyến tính", $C=1$). Hoặc chúng ta có thể sử dụng lớp SGDClassifier, với SGDClassifier(loss="hinge", $\alpha=1/(m*C)$). Điều này áp dụng Stochastic Gradient Descent thông thường (xem Chương 4) để training classifier SVM tuyến tính. Nó không hội tụ nhanh như lớp LinearSVC, nhưng nó có thể hữu ích để xử lý các tác vụ classification trực tuyến hoặc datasets khổng lồ không vừa với bộ nhớ (training ngoài lõi).

Lớp LinearSVC chuẩn hóa số hạng bias, vì vậy trước tiên bạn nên căn giữa training

set bằng cách trừ đi giá trị trung bình của nó. Điều này là tự động nếu bạn chia tỷ lệ dữ liệu bằng StandardScaler. Ngoài ra, hãy đảm bảo bạn đặt tổn thất hyperparameter thành "bản lề", vì đây không phải là giá trị mặc định. Cuối cùng, để có hiệu suất tốt hơn, bạn nên đặt hyperparameter kép thành Sai, trừ khi có nhiều features hơn phiên bản training (chúng ta sẽ thảo luận về tính đối ngẫu ở phần sau của chương).

156 | Chương 5: Máy vector hỗ trợ

1.4 Classification SVM phi tuyến

Mặc dù các classifier SVM tuyến tính hoạt động hiệu quả và hoạt động tốt một cách đáng ngạc nhiên trong nhiều trường hợp, nhiều datasets thậm chí còn không thể phân tách tuyến tính được. Một cách tiếp cận để xử lý datasets phi tuyến là thêm nhiều features hơn, chẳng hạn như features đa thức (như bạn đã làm trong Chương 4); trong một số trường hợp, điều này có thể dẫn đến dataset có thể phân tách tuyến tính. Hãy xem xét biểu đồ bên trái trong Hình 5-5: nó thể hiện một dataset đơn giản chỉ có một tính năng, x_1 . Dataset này không thể phân tách tuyến tính như bạn có thể thấy. Nhưng nếu bạn thêm feature $x_2 = (x_1)^2$ thứ hai, thì dataset 2D thu được hoàn toàn có thể phân tách tuyến tính.

Để thực hiện ý tưởng này bằng cách sử dụng Scikit-Learn, hãy tạo một Pipeline chứa PolynomialFeatures transformer (được thảo luận trong "Polynomial Regression" trên trang 128), tiếp theo là StandardScaler và LinearSVC. Hãy kiểm tra điều này trên các mặt trăng dataset: đây là đồ chơi dataset dành cho classification nhị phân trong đó các điểm dữ liệu có hình dạng như hai nửa vòng tròn đan xen (xem Hình 5-6). Bạn có thể tạo dataset này bằng hàm `make_moons()`:

1.5 Hạt nhân đa thức

Việc thêm đa thức features rất đơn giản để thực hiện và có thể hoạt động hiệu quả với tất cả các loại thuật toán Machine Learning (không chỉ SVMs). Điều đó có nghĩa là, ở mức độ đa thức thấp, phương pháp này không thể xử lý datasets rất phức tạp và với mức độ đa thức cao, nó tạo ra một số lượng lớn features, khiến model quá chậm.

May mắn thay, khi sử dụng SVMs, bạn có thể áp dụng một kỹ thuật toán học gần như kỳ diệu được gọi là thủ thuật kernel (được giải thích sau). Thủ thuật kernel giúp bạn có thể nhận được kết quả tương tự như khi bạn đã thêm nhiều features đa thức, ngay cả với các đa thức bậc rất cao mà không thực sự cần phải thêm chúng. Vì vậy, không có sự bùng nổ tổ hợp về số lượng features vì bạn không thực sự thêm bất kỳ features nào. Thủ thuật này được thực hiện bởi lớp SVC. Hãy thử nghiệm nó trên mặt trăng dataset:

Code này training classifier SVM bằng cách sử dụng hạt nhân đa thức bậc ba. Nó được thể hiện ở bên trái trong Hình 5-7. Bên phải là một classifier SVM khác sử dụng hạt nhân đa thức bậc 10. Rõ ràng, nếu model của bạn là overfitting, bạn có thể muốn giảm bậc đa thức. Ngược lại, nếu là underfitting, bạn có thể thử tăng nó lên. Hyperparameter `coef0` kiểm soát mức độ ảnh hưởng của model bởi đa thức bậc cao so với đa thức bậc thấp.

158 | Chương 5: Máy vector hỗ trợ

Một cách tiếp cận phổ biến để tìm các giá trị hyperparameter phù hợp là sử dụng grid search (xem Chương 2). Thông thường sẽ nhanh hơn nếu thực hiện grid search rất thô trước tiên, sau đó là grid search mịn hơn xung quanh các giá trị tốt nhất được tìm thấy.

Việc hiểu rõ những gì mỗi hyperparameter thực sự làm cũng có thể giúp bạn tìm kiếm ở phần bên phải của không gian siêu tham số.

1.6 Tính năng tương tự

Một kỹ thuật khác để giải quyết các vấn đề phi tuyến là thêm features được tính toán bằng hàm tương tự, hàm này đo mức độ giống nhau của mỗi phiên bản với một mốc cụ thể. Ví dụ: hãy lấy dataset 1D đã thảo luận trước đó và thêm hai điểm mốc vào nó tại $x_1 = -2$ và $x_2 = 1$ (xem biểu đồ bên trái trong Hình 5-8). Tiếp theo, hãy xác định hàm tương tự là Hàm cơ sở bán kính Gaussian (RBF) với $\gamma = 0,3$ (xem Công thức 5-1).

Phương trình 5-1. Gaussian RBF
$$x_i = \exp(-\gamma \|x_i - x_c\|^2)$$

Đây là hàm hình chuông thay đổi từ 0 (rất xa mốc) đến 1 (tại mốc). Bây giờ chúng ta đã sẵn sàng tính features mới. Ví dụ: hãy xem ví dụ $x_1 = -1$: nó nằm ở khoảng cách 1 từ mốc đầu tiên và 2 từ mốc thứ hai. Do đó, features mới của nó là $x_2 = \exp(-0,3 \times 12) = 0,74$ và $x_3 = \exp(-0,3 \times 22) = 0,30$. Biểu đồ bên phải trong Hình 5-8 cho thấy dataset đã được chuyển đổi (bỏ features ban đầu). Như bạn có thể thấy, bây giờ nó có thể phân tách tuyến tính.

Bạn có thể tự hỏi làm thế nào để chọn các điểm mốc. Cách tiếp cận đơn giản nhất là tạo một điểm mốc tại vị trí của từng phiên bản trong dataset. Làm như vậy sẽ tạo ra nhiều chiều và do đó làm tăng khả năng training set được chuyển đổi sẽ có thể phân tách tuyến tính. Nhược điểm là training set với m phiên bản và n features được chuyển thành training set với m phiên bản và m features (giả sử bạn bỏ features ban đầu). Nếu training set của bạn rất lớn thì bạn sẽ có số lượng features lớn tương đương.

1.7 Hạt nhân Gaussian RBF

Giống như phương pháp features đa thức, phương pháp features tương tự có thể hữu ích với bất kỳ thuật toán Machine Learning nào, nhưng việc tính toán tất cả features bổ sung có thể tốn kém về mặt tính toán, đặc biệt là trên các tập training lớn. Một lần nữa, thủ thuật kernel thực hiện phép thuật SVM của nó, giúp bạn có thể thu được kết quả tương tự như thể bạn đã thêm nhiều features tương tự. Hãy thử lớp SVC với kernel Gaussian RBF:

Model này được thể hiện ở phía dưới bên trái trong Hình 5-9. Các biểu đồ khác hiển thị các model được training với các giá trị khác nhau của gamma hyperparameters (γ) và C . Gamma tăng làm cho đường cong hình chuông hẹp hơn (xem các biểu đồ bên trái trong Hình 5-8). Kết quả là, phạm vi ảnh hưởng của mỗi trường hợp nhỏ hơn: ranh giới quyết định cuối cùng trở nên bất thường hơn, dao động xung quanh các trường hợp riêng lẻ. Ngược lại, giá trị gamma nhỏ làm cho đường cong hình chuông rộng hơn: các trường hợp có phạm vi ảnh hưởng lớn hơn và ranh giới quyết định sẽ trở nên mượt mà hơn. Đây hoạt động như một regularization

160 | Chương 5: Máy vector hỗ trợ

Hyperparameter: nếu model của bạn là overfitting, bạn nên giảm nó; nếu là underfitting thì bạn nên tăng lên (tương tự như C hyperparameter).

Các hạt nhân khác tồn tại nhưng hiếm khi được sử dụng hơn. Một số hạt nhân được chuyên dùng cho các cấu trúc dữ liệu cụ thể. Hạt nhân chuỗi đôi khi được sử dụng khi classification tài liệu văn bản hoặc chuỗi DNA (ví dụ: sử dụng hạt nhân chuỗi con hoặc hạt nhân dựa trên khoảng cách Levenshtein).

Với rất nhiều hạt nhân để lựa chọn, làm thế nào bạn có thể quyết định nên sử dụng hạt nhân nào? Theo nguyên tắc chung, trước tiên bạn phải luôn thử hạt nhân tuyến tính (hãy nhớ rằng LinearSVC nhanh hơn nhiều so với SVC(kernel="tuyến tính")), đặc biệt nếu training set rất lớn hoặc nếu nó có nhiều features. Nếu training set không quá lớn, bạn cũng nên thử kernel Gaussian RBF; nó hoạt động tốt trong hầu hết các trường hợp. Sau đó, nếu bạn có thời gian rảnh và khả năng tính toán, bạn có thể thử nghiệm với một số hạt nhân khác, sử dụng cross-validation và grid search. Bạn muốn thử nghiệm như vậy, đặc biệt nếu có hạt nhân chuyên dụng cho cấu trúc dữ liệu training set của bạn.

1.8 Độ phức tạp tính toán

Lớp LinearSVC dựa trên thư viện liblinear, thư viện này triển khai thuật toán được tối ưu hóa cho SVMs tuyến tính.¹ Nó không hỗ trợ thủ thuật kernel, nhưng nó có quy mô gần như tuyến tính với số lượng phiên bản training và số lượng features. Độ phức tạp về thời gian training của nó là khoảng $O(m \times n)$.

Thuật toán sẽ mất nhiều thời gian hơn nếu bạn yêu cầu precision rất cao. Điều này được kiểm soát bởi dung sai hyperparameter (được gọi là tol trong Scikit-Learn). Trong hầu hết các tác vụ classification, dung sai mặc định là ổn.

Lớp SVC dựa trên thư viện libsvm, thực hiện một thuật toán hỗ trợ thủ thuật kernel.² Độ phức tạp về thời gian training thường nằm trong khoảng $O(m^2 \times n)$ và $O(m^3 \times n)$. Thật không may, điều này có nghĩa là nó sẽ cực kỳ chậm khi số lượng phiên bản training trở nên lớn (ví dụ: hàng trăm nghìn phiên bản). Thuật toán này hoàn hảo cho các tập training cỡ nhỏ hoặc vừa phức tạp. Nó có tỷ lệ phù hợp với số lượng features, đặc biệt là với features thừa thớt (tức là khi mỗi phiên bản có ít features khác 0). Trong trường hợp này, thuật toán có tỷ lệ gần đúng với số features khác 0 trung bình trên mỗi phiên bản. Bảng 5-1 so sánh các lớp SVM classification của Scikit-Learn.

Lớp Độ phức tạp về thời gian Hỗ trợ ngoài lõi Cần mở rộng quy mô Thủ thuật hạt nhân

LinearSVC	$O(m \times n)$	Không	Có	Không
SGDClassifier	$O(m \times n)$	Có	Có	Không
SVC	$O(m^2 \times n)$ đến $O(m^3 \times n)$	Không	Có	Có

1.9 Regression SVM

Như đã đề cập trước đó, thuật toán SVM rất linh hoạt: nó không chỉ hỗ trợ classification tuyến tính và phi tuyến tính mà còn hỗ trợ regression tuyến tính và phi tuyến tính. Để sử dụng SVMs cho regression thay vì classification, mẹo là đảo ngược mục tiêu: thay vì cố gắng khớp đường phổ lớn nhất có thể giữa hai lớp trong khi hạn chế vi phạm lề, SVM Regression cố gắng khớp càng nhiều phiên bản càng tốt trên đường phổ trong khi hạn chế vi phạm lề (tức là các trường hợp ngoài đường). Chiều rộng của đường phổ được điều khiển bởi hyperparameter, C . Hình 5-10 thể hiện hai SVM tuyến tính

162 | Chương 5: Máy vector hỗ trợ

Regression models được training trên một số dữ liệu tuyến tính ngẫu nhiên, một dữ liệu có biên độ lớn ($\sigma = 1,5$) và dữ liệu còn lại có biên độ nhỏ ($\sigma = 0,5$).

Việc thêm nhiều phiên bản training hơn vào lề không ảnh hưởng đến dự đoán của model; do đó, model được cho là không nhạy cảm.

Bạn có thể sử dụng lớp LinearSVR của Scikit-Learn để thực hiện SVM Regression tuyến tính. Đoạn code sau tạo ra model được biểu thị ở bên trái trong Hình 5-10 (dữ liệu

training phải được chia tỷ lệ và căn giữa trước):

Để giải quyết các tác vụ regression phi tuyến tính, bạn có thể sử dụng SVM model được kernel hóa. Hình 5-11 thể hiện SVM Regression trên training set bậc hai ngẫu nhiên, sử dụng hạt nhân đa thức bậc hai. Có rất ít regularization ở ô bên trái (tức là giá trị C lớn) và nhiều regularization hơn ở ô bên phải (tức là giá trị C nhỏ).

Đoạn code sau sử dụng lớp SVR của Scikit-Learn (hỗ trợ thủ thuật kernel) để tạo ra model được biểu thị ở bên trái trong Hình 5-11:

SVMs cũng có thể được sử dụng để phát hiện ngoại lệ; xem tài liệu của Scikit-Learn để biết thêm chi tiết.

1.10 Dưới mui xe

Phần này giải thích cách SVMs đưa ra dự đoán và cách hoạt động của các thuật toán training của chúng, bắt đầu với các classifier SVM tuyến tính. Nếu bạn mới bắt đầu với Machine Learning, bạn có thể bỏ qua nó một cách an toàn và đi thẳng đến các bài tập ở cuối chương này và quay lại sau khi bạn muốn hiểu sâu hơn về SVMs.

Đầu tiên, một lời về ký hiệu. Trong Chương 4, chúng ta đã sử dụng quy ước đặt tất cả model parameters vào một vector $\mathbf{w}$, bao gồm số hạng bias w_0 và đầu vào feature weights w_1 vào w_n , đồng thời thêm đầu vào bias $x_0 = 1$ cho tất cả các phiên bản. Trong chương này chúng ta sẽ sử dụng một quy ước thuận tiện hơn (và phổ biến hơn) khi giải quyết các vấn đề

164 | Chương 5: Máy vector hỗ trợ

SVMs: thuật ngữ bias sẽ được gọi là b và vector feature weights sẽ được gọi là $\mathbf{w}$. Không có bias feature nào sẽ được thêm vào các vector feature đầu vào.

1.11 Chức năng quyết định và dự đoán

Classifier SVM tuyến tính model dự đoán lớp của một phiên bản $\mathbf{x}$ mới bằng cách tính toán hàm quyết định $w \cdot \mathbf{x} + b = w_1 x_1 + \dots + w_n x_n + b$. Nếu kết quả là dương, thì lớp dự đoán $\hat{y}$ là lớp dương (1), còn nếu không thì nó là lớp âm (0); xem phương trình 5-2.

Phương trình 5-2. Dự đoán classification SVM tuyến tính $y = 0$ nếu $w \cdot \mathbf{x} + b < 0$,
1 nếu $w \cdot \mathbf{x} + b \geq 0$

Các đường đứt nét biểu thị các điểm mà hàm quyết định bằng 1 hoặc -1: chúng song song và ở khoảng cách bằng nhau với ranh giới quyết định và chúng tạo thành một lề xung quanh nó. Training classifier SVM tuyến tính có nghĩa là tìm các giá trị của $\mathbf{w}$ và b làm cho lề này càng rộng càng tốt trong khi tránh vi phạm lề (lề cứng) hoặc hạn chế chúng (lề mềm).

1.12 Mục tiêu training

Xét gradient của hàm quyết định: nó bằng chuẩn của vector weight, $\|\mathbf{w}\|$. Nếu chúng ta chia gradient này cho 2 thì những điểm mà hàm quyết định bằng ± 1 sẽ cách xa ranh giới quyết định gấp đôi. Nói cách khác, chia gradient cho 2 sẽ nhân số lề với 2. Điều này có thể dễ hình dung hơn ở dạng 2D, như trong Hình 5-13. Vector weight $\mathbf{w}$ càng nhỏ thì lề càng lớn.

Vì vậy, chúng ta muốn giảm thiểu $\|\mathbf{w}\|$ để có được mức chênh lệch lớn. Nếu chúng ta cũng muốn tránh bất kỳ vi phạm ký quỹ nào (lề cứng), thì chúng ta cần hàm quyết định lớn hơn 1 đối với tất cả các trường hợp training tích cực và thấp hơn -1 đối với các

trường hợp training tiêu cực. Nếu chúng ta xác định $t(i) = -1$ cho các trường hợp phủ định (nếu $y(i) = 0$) và $t(i) = 1$ cho các trường hợp dương (nếu $y(i) = 1$), thì chúng ta có thể biểu thị ràng buộc này dưới dạng $t(i)(w \cdot x(i) + b) \geq 1$ cho tất cả các trường hợp.

Do đó, chúng ta có thể biểu diễn mục tiêu của classifier SVM tuyến tính biên cứng như bài toán tối ưu hóa có ràng buộc trong Công thức 5-3.

Phương trình 5-3. Mục tiêu classification SVM tuyến tính lề cứng giảm thiểu w , b
 $\min_{w, b} \sum_{i=1}^m (t_i w \cdot x_i + b)$ với $i = 1, 2, \dots, m$

166 | Chương 5: Máy vector hỗ trợ

Chúng ta đang giảm thiểu $\frac{1}{2} \|w\|^2$, bằng $\frac{1}{2} \|w\|^2$, thay vì giảm thiểu $\|w\|$. Thật vậy, $\frac{1}{2} \|w\|^2$ có đạo hàm đơn giản, đẹp mắt (nó chỉ là w), trong khi $\|w\|$ không khả vi tại $w = 0$. Các thuật toán tối ưu hóa hoạt động tốt hơn nhiều trên các hàm khả vi.

Để đạt được mục tiêu biên độ mềm, chúng ta cần đưa ra một biến chùng (i) cho mỗi trường hợp: (i) đo lường mức độ trường hợp thứ i được phép vi phạm biên. Bây giờ chúng ta có hai mục tiêu trái ngược nhau: tạo ra các biến phụ trợ càng nhỏ càng tốt để giảm vi phạm biên và tạo $\frac{1}{2} \|w\|^2$ càng nhỏ càng tốt để tăng biên. Đây là lúc C hyperparameter xuất hiện: nó cho phép chúng ta xác định sự cân bằng giữa hai mục tiêu này. Điều này cho chúng ta bài toán tối ưu hóa bị ràng buộc trong phương trình 5-4.

Phương trình 5-4. Mục tiêu classification SVM tuyến tính lề mềm giảm thiểu w , b ,
 $\min_{w, b, C} \sum_{i=1}^m (t_i w \cdot x_i + b) + C \sum_{i=1}^m \xi_i$ với $i = 1, 2, \dots, m$

1.13 Lập trình bậc hai

Các bài toán về lề cứng và lề mềm đều là các bài toán tối ưu bậc hai lỗi với các ràng buộc tuyến tính. Những bài toán như vậy được gọi là bài toán quy hoạch bậc hai (QP). Có sẵn nhiều bộ giải có sẵn để giải các bài toán QP bằng cách sử dụng nhiều kỹ thuật khác nhau nằm ngoài phạm vi của cuốn sách này.⁵

Việc xây dựng bài toán tổng quát được đưa ra bởi phương trình 5-5.

Phương trình 5-5. Bài toán lập trình bậc hai

Giảm thiểu p

$\min_p H p + f$ tuân theo $A p \leq b$ trong đó p là vector n_p chiều ($n_p =$ số parameters),

H là matrix $n_p \times n_p$, f là vector n_p chiều,

A là matrix $n_c \times n_p$ ($n_c =$ số ràng buộc), b là vector n_c chiều.

Bạn có thể dễ dàng xác minh rằng nếu bạn đặt QP parameters theo cách sau, bạn sẽ nhận được mục tiêu classification SVM tuyến tính lề cứng:

- $n_p = n + 1$, trong đó n là số features (+1 dành cho số hạng bias).
- $n_c = m$, trong đó m là số mẫu training.
- H là matrix nhận dạng $n_p \times n_p$, ngoại trừ số 0 ở ô trên cùng bên trái (bỏ qua thuật ngữ bias).
- $f = 0$, một vector n_p chiều chứa đầy các số 0.
- $b = -1$, một vector n_c chiều chứa đầy -1.
- $a(i) = -t(i) x'(i)$, trong đó $x'(i)$ bằng $x(i)$ có thêm bias feature $x'_0 = 1$.

Một cách để training classifier SVM tuyến tính lề cứng là sử dụng bộ giải QP có sẵn và chuyển nó cho parameters trước đó. Vector kết quả p sẽ chứa số hạng bias $b = p_0$ và feature weights $w_i = p_i$ với $i = 1, 2, \dots, n$. Tương tự, bạn có thể sử dụng bộ giải QP để giải bài toán lề mềm (xem bài tập ở cuối chương).

Để sử dụng thủ thuật kernel, chúng ta sẽ xem xét một vấn đề tối ưu hóa có ràng buộc khác.

1.14 Vấn đề kép

Cho một bài toán tối ưu hóa bị ràng buộc, được gọi là bài toán cơ bản, có thể biểu diễn một bài toán khác nhưng có liên quan chặt chẽ, được gọi là bài toán kép của nó. Các

168 | Chương 5: Máy vector hỗ trợ

Lời giải cho bài toán đối ngẫu thường đưa ra giới hạn dưới cho lời giải của bài toán cơ bản, nhưng trong một số điều kiện, nó có thể có cùng nghiệm với bài toán cơ bản. May mắn thay, bài toán SVM xảy ra đáp ứng các điều kiện này,⁶ vì vậy bạn có thể chọn giải bài toán cơ bản hoặc bài toán kép; cả hai sẽ có cùng một giải pháp. Phương trình 5-6 cho thấy dạng đối ngẫu của mục tiêu SVM tuyến tính (nếu bạn muốn biết cách rút ra bài toán đối ngẫu từ bài toán cơ bản, hãy xem Phụ lục C).

Phương trình 5-6. Dạng kép của mục tiêu SVM tuyến tính cực tiểu hóa

$$\min_{\alpha} \sum_{i=1}^m \sum_{j=1}^m \alpha_{ij} t_i t_j x_i \cdot x_j - \sum_{i=1}^m \alpha_i \quad \text{tuân theo} \quad \alpha_i \geq 0 \quad \text{vì } i = 1, 2, \dots, m$$

Khi bạn tìm thấy vector làm cực tiểu hóa phương trình này (sử dụng bộ giải QP), hãy sử dụng Công thức 5-7 để tính w và b nhằm cực tiểu hóa bài toán cơ bản.

Phương trình 5-7. Từ nghiệm kép đến nghiệm gốc $w = \sum_{i=1}^m \alpha_i t_i x_i$ và $b = \frac{1}{n} \sum_{i=1}^n \alpha_i$ với $\alpha_i > 0$ và $t_i = -w \cdot x_i$

Bài toán kép được giải quyết nhanh hơn bài toán cơ bản khi số lượng phiên bản training nhỏ hơn số lượng features. Quan trọng hơn, vấn đề kép khiến cho kernel có thể thực hiện được thủ thuật, trong khi kernel nguyên thủy thì không. Vậy thủ thuật kernel này là gì?

1.15 Hạt nhân SVMs

Giả sử bạn muốn áp dụng phép biến đổi đa thức bậc hai cho training set hai chiều (chẳng hạn như training set của mặt trăng), sau đó training classifier SVM tuyến tính trên training set đã biến đổi. Phương trình 5-8 cho thấy hàm ánh xạ đa thức bậc hai mà bạn muốn áp dụng.

$$\begin{bmatrix} x_1^2 & x_1 x_2 & x_2^2 \end{bmatrix}$$

Lưu ý rằng vector được chuyển đổi là 3D thay vì 2D. Bây giờ chúng ta hãy xem điều gì xảy ra với một vài vector 2D, a và b , nếu chúng ta áp dụng ánh xạ đa thức bậc hai này và sau đó tính tích số chấm⁷ của các vector đã biến đổi (Xem Công thức 5-9).

Phương trình 5-9. Thủ thuật hạt nhân để ánh xạ đa thức bậc hai $a \cdot b = a^T b$

$$\begin{bmatrix} a_1^2 & a_1 a_2 & a_2^2 \end{bmatrix} \cdot \begin{bmatrix} b_1^2 \\ b_1 b_2 \\ b_2^2 \end{bmatrix}$$

$$= a_1^2 b_1^2 + 2 a_1 a_2 b_1 b_2 + a_2^2 b_2^2$$

$$= (a_1 b_1 + a_2 b_2)^2$$

$$= (a \cdot b)^2$$

$$= a \cdot b$$

Còn chuyện đó thì sao? Tích vô hướng của các vector biến đổi bằng bình phương tích vô hướng của các vector gốc: $(a \cdot b)^2 = (a \cdot b)^2$.

Đây là thông tin quan trọng: nếu bạn áp dụng phép biến đổi cho tất cả các trường hợp training thì bài toán kép (xem Công thức 5-6) sẽ chứa tích chấm $(x(i) \cdot x(j))$. Nhưng nếu là phép biến đổi đa thức bậc hai được xác định trong Công thức 5-8, thì bạn có thể thay thế tích vô hướng này của các vector biến đổi bằng $x(i) \cdot x(j)$.

Vì vậy, bạn không cần phải chuyển đổi các trường hợp training; chỉ cần thay tích số chấm bằng bình phương của nó trong Công thức 5-6. Kết quả sẽ hoàn toàn giống như khi

bạn gặp khó khăn trong việc biến đổi training set sau đó khớp với thuật toán SVM tuyến tính, nhưng thủ thuật này làm cho toàn bộ quá trình tính toán hiệu quả hơn nhiều.

Hàm $K(a, b) = (a - b)^2$ là hạt nhân đa thức bậc hai. Trong Machine Learning, hạt nhân là một hàm có khả năng tính toán tích chập $(a) \cdot (b)$,

170 | Chương 5: Máy vector hỗ trợ